ORBITGUARD — Build Specification

KLE Tech Hackfest 2026 · Challenge 1C: Space Debris Risk Prediction

0. How to use this document (read first, Claude Code)
1. The problem in one page
2. What we are building
3. Architecture
4. Repository layout and environment
5. Data sources
6. Module 1 — Event-level dataset (`features.py`)
7. Module 2 — Models and calibration (`models.py`, `calibrate.py`)
8. Module 3 — Evaluation (`evaluate.py`)
9. Module 4 — Explainability and briefing (`explain.py`)
10. Module 5 — Collision probability and maneuver planner (`pc.py`, `cw.py`, `maneuver.py`)
11. Module 6 — Live screener (`screen.py`)
12. Module 7 — Dashboard (`web/`)
13. API contract (FastAPI, `api/main.py`)
14. Tests (`tests/`, run with `pytest`)
15. Build order (solo developer + Claude Code)
16. Demo script (≈ 5 minutes)
17. Known limitations (say them before the judges do)
18. References

0. How to use this document (read first, Claude Code)

This is the complete build spec for a hackathon project. You (Claude Code) will build it inside an empty project folder. Follow these rules:

1. **Read the whole document before writing code.** Sections 6–12 are the modules; Section 13 is the API contract that connects them; Section 15 is the build order.
2. **Build in milestone order (Section 15).** Every milestone ends with a runnable, demoable state. Never leave the repo broken.
3. **Mock first, then real.** The backend must serve mock JSON matching Section 13 from the first hour, so the frontend can be built in parallel. Replace mocks module by module.
4. **The demo must never crash.** Every live dependency (CelesTrak, LLM API) needs a cached or template fallback.
5. **Respect CelesTrak's rate limits (Section 5.2).** Cache downloads to disk and never re-download a file younger than 2 hours. Stop on any non-200 response. Repeated requests get the team's IP firewalled.
6. **No data leakage.** Features for an event may only use CDMs with `time_to_tca ≥ cutoff` (default 2 days). Train/validation/test splits are by `event_id`, never by row. Tests in Section 14 enforce this.
7. **Write tests for the math** (Pc, Clohessy–Wiltshire, leakage) as specified in Section 14, and run them.
8. Keep code simple, typed where cheap, and commented where the physics is non-obvious. Prefer working over clever.
9. Log every metric to `artifacts/metrics.json` so the dashboard and the pitch deck use the same numbers.

Project name used throughout: **OrbitGuard**. Rename freely.

1. The problem in one page

Context. Low Earth Orbit holds tens of thousands of tracked objects moving at about 7.5 km/s. Even a 1 cm fragment carries enough energy to disable a satellite. When two objects are predicted to pass close, the satellite operator receives a stream of **Conjunction Data Messages (CDMs)** over the days before the **Time of Closest Approach (TCA)**. Each CDM carries a **Probability of Collision (PoC)**, and that number changes noisily as tracking improves.

The operator's dilemma. Operators get hundreds of alerts a week and almost all are noise. Maneuvering burns fuel, shortens mission life and interrupts service. Not maneuvering when the threat is real can destroy the asset and create a debris cloud. Operators typically decide about 2 days before TCA, using uncertain data.

Mission (from the brief). Automate the **Go / No-Go collision decision** in LEO.

Target (from the brief). Build an AI that:

- parses noisy orbital data (TLE catalogs, CDMs, space weather),
- delivers **calibrated confidence** (PoC with confidence bands),
- outputs **ranked alerts**,
- recommends **fuel-efficient avoidance maneuvers** (Δv in m/s),
- **explains why** each alert fired (SHAP), because operators won't act on a black box.

Judging KPIs (from the brief) and how we measure each:

KPI	Brief's wording	Our measurement
Calibration	Does a 1-in-10,000 predicted risk happen 1-in-10,000 times?	Brier score + reliability diagram on held-out events; P10–P90 band coverage
Hit/Miss rate	>95% recall on high-risk events while dropping false alarms	Recall and false-alarm rate at the chosen threshold, vs the naive baseline
Lead time	Flag genuine risks 3–7 days before TCA	Recall and F2 at feature cutoffs of 2, 3, 4, 5, 7 days
Throughput	Screen the full object catalog in minutes	Wall-clock time to screen one asset vs the whole catalog over 72 h (logged)

Personas we serve (from the brief):

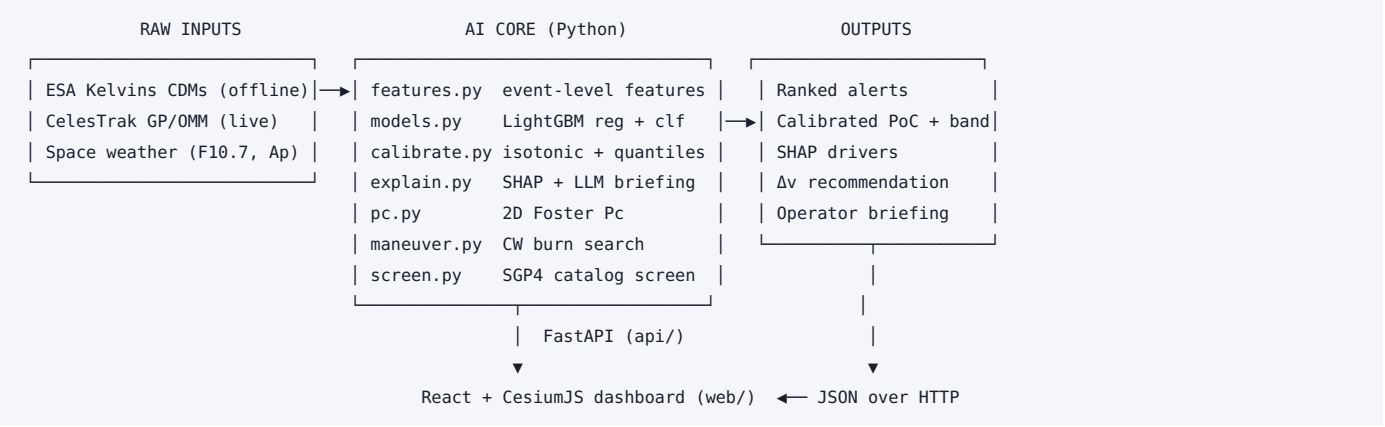
- *Satellite operations engineer* → Go/No-Go decision and Δv option.
- *Orbital risk analyst* → calibrated risk with confidence band, traceable features (SHAP).
- *Space safety regulator* → exportable, transparent risk report.
- *Mission designer* (lightly) → orbit-risk comparison is out of scope; mention as future work.

2. What we are building

Seven components:

1. **Data pipeline** — loads ESA Kelvins CDMs, builds one row per event using only CDMs available before the cutoff.
2. **Risk model** — LightGBM models that predict the event's **final risk** (log10 PoC at the last CDM) and the probability that it is **high-risk** (final PoC ≥ 10^-6).
3. **Calibration and confidence bands** — isotonic calibration of the high-risk probability; quantile models give P10/P50/P90 of final risk.
4. **Explainer** — SHAP top drivers per event, plus an LLM-written operator briefing in ALERT / DIAGNOSTIC / ACTION form (template fallback).
5. **Pc engine + maneuver planner** — computes 2D collision probability from geometry and covariance; searches burn size and timing with Clohessy–Wiltshire relative motion; outputs the risk-vs-Δv Pareto front and the cheapest safe burn.
6. **Live screener** — downloads the live catalog from CelesTrak, propagates it with SGP4, finds close approaches for a chosen asset (e.g. ISS) over the next 72 h, and times itself.
7. **Mission-control dashboard** — 3D globe, ranked alert list, alert detail (CDM history, risk band, SHAP, briefing, maneuver chart), CDM replay slider, metrics page.

3. Architecture



4. Repository layout and environment

```
orbitguard/
├── README.md                 # how to run; team; demo script
├── CLAUDE.md                 # one line: "Follow SPEC.md. Build in Section 15 order."
├── SPEC.md                   # this document
├── requirements.txt
├── Makefile                  # make data | train | eval | api | web | test
├── data/
│   ├── raw/                  # downloads (gitignored)
│   └── processed/            # parquet event tables (gitignored)
├── artifacts/                # models, metrics.json, plots, demo_events.json
├── orbitguard/               # Python package
│   ├── config.py             # paths, thresholds, constants
│   ├── download.py           # Kelvins + CelesTrak + space weather (cached)
│   ├── features.py           # event-level dataset builder
│   ├── models.py             # train baseline, regressor, classifier, quantiles
│   ├── calibrate.py          # isotonic calibration, threshold selection
│   ├── evaluate.py           # all metrics + plots → artifacts/
│   ├── explain.py            # SHAP + briefing (LLM + template)
│   ├── pc.py                 # collision probability
│   ├── cw.py                 # Clohessy–Wiltshire relative motion
│   ├── maneuver.py           # burn search, Pareto front, propellant
│   ├── screen.py             # SGP4 catalog screening
│   └── decision.py           # Go / No-Go / Watch logic
├── api/
│   ├── main.py               # FastAPI app
│   ├── schemas.py            # Pydantic models = Section 13 contract
│   └── mock/                 # mock JSON responses (milestone 0)
├── web/                      # React + Vite + CesiumJS
└── tests/                    # pytest (Section 14)
```

Python 3.11. `requirements.txt`:

```
pandas
numpy
scipy
pyarrow
scikit-learn
lightgbm
shap
matplotlib
sgp4>=2.23
requests
fastapi
uvicorn[standard]
pydantic>=2
python-dotenv
pytest
anthropic          # optional, for LLM briefing
```

Frontend: Node 20, Vite + React + TypeScript, `cesium` + `vite-plugin-cesium`, `recharts`, `axios`.

No GPU needed. Everything runs on a laptop CPU with 8 GB RAM.

Environment variables (`.env`, never committed):

```
LLM_PROVIDER=anthropic          # or "none" to force template briefings
ANTHROPIC_API_KEY=...
LLM_MODEL=claude-haiku-4-5-20251001
CESIUM_ION_TOKEN=               # optional; the app must work without it
```

5. Data sources

5.1 ESA Kelvins Collision Avoidance Challenge (training data)

- **Download:** Zenodo record 4463683 (`https://zenodo.org/records/4463683`, file `Collision Avoidance Challenge - Dataset.zip`). Mirrors on the Kelvins site: `https://kelvins.esa.int/media/public/competitions/collision-avoidance-challenge/train_data.zip` and `.../test_data.csv`. If automatic download fails, print a clear message asking the user to download manually into `data/raw/kelvins/`.
- **Size:** train ≈ 162,634 rows / 13,154 events; test ≈ 24,484 rows / 2,167 events.
- **Structure:** one row = one CDM. Rows are grouped by `event_id`; within an event, CDMs are ordered by decreasing `time_to_tca` (days).
- **Target:** `risk` = log10 of collision probability. For each event, the **final risk** is the `risk` of the CDM with the smallest `time_to_tca`.
- **The 2-day rule:** the original competition only allowed CDMs at least 2 days before TCA as inputs, because the maneuver decision is made around then.
- **High-risk class:** final risk ≥ −6 (i.e. PoC ≥ 10^-6). This is the official class boundary. It is rare.
- **Test set:** hand-picked to over-represent high-risk events; its events have the last CDM within 1 day of TCA. **Check on load:** if `test_data.csv` contains CDMs with `time_to_tca < 2`, the ground truth is present → use it as the final held-out test set (target = last CDM risk; inputs = CDMs ≥ cutoff). If it does not, carve the test set from train events instead.
- **Official competition score** (implement it too, for credibility): L = MSE_HR / F2, where F2 is computed over all events using the −6 boundary and MSE is computed only on events whose true final risk ≥ −6. Lower is better.

Key columns (103 total; all numeric except `c_object_type`). Prefix `t_` = target (ESA satellite), `c_` = chaser (debris / other object).

Column	Meaning
`event_id`, `mission_id`	event and mission identifiers
`time_to_tca`	days from CDM creation to TCA
`risk`	log10 PoC computed at that CDM
`max_risk_estimate`, `max_risk_scaling`	max PoC obtained by scaling the combined covariance; scaling factor
`miss_distance` (m), `relative_speed` (m/s)	geometry at TCA
`relative_position_r/t/n` (m)	relative position: radial / transverse (along-track) / normal (cross-track)
`relative_velocity_r/t/n` (m/s)	relative velocity, same frame
`c_object_type`	chaser type (categorical)
`geocentric_latitude`, `azimuth`, `elevation`	conjunction point and relative-velocity angles
`F10`, `F3M`, `AP`, `SSN`	space weather: F10.7 flux, its 81-day mean, Ap index, sunspot number
`x_sigma_r/t/n` (m)	position standard deviations (x = t or c)
`x_sigma_rdot/tdot/ndot` (m/s)	velocity standard deviations
`x_ct_r`, `x_cn_r`, `x_cn_t`, …	covariance correlation coefficients
`x_position_covariance_det`	determinant of position covariance
`x_cd_area_over_mass`, `x_cr_area_over_mass`	drag and solar-radiation coefficients × A/m
`x_h_apo`, `x_h_per` (km), `x_ecc`, `x_j2k_inc`, `x_j2k_sma`	orbit shape
`x_span` (m)	object size used in PoC (chaser minimum 2 m)
`x_obs_available`, `x_obs_used`, `x_weighted_rms`, `x_residuals_accepted`, `x_actual_od_span`, `x_recommended_od_span`, `x_time_lastob_start/end`, `x_sedr`, `x_rcs_estimate`	orbit-determination quality fields

Data-cleaning notes:

- Some rows have missing covariance values. Leave NaN for LightGBM (handles it natively); drop such rows only for the Pc engine.
- `risk` has a very negative floor for negligible events. Inspect `risk.min()`, then clip all predictions to the observed `[min, max]` range.
- Remove events with a single CDM from training of trend features (keep them with trend features = NaN).

5.2 CelesTrak (live orbital catalog)

- **Query format:** `https://celestrak.org/NORAD/elements/gp.php?GROUP=<group>&FORMAT=csv` (also `CATNR=<number>`). Always use the **.org** domain.
- **Use OMM CSV or JSON, never TLE text.** New objects now have 6-digit catalog numbers that the TLE format cannot represent. Parse OMM rows with `sgp4.omm` (`from sgp4 import omm; from sgp4.api import Satrec; sat = Satrec(); omm.initialize(sat, fields)`).
- **Groups to use:** `active` (payloads), `stations` (ISS etc.), and debris groups such as `cosmos-1408-debris`, `fengyun-1c-debris`,

`iridium-33-debris`, `cosmos-2251-debris`. If a group returns non-200, skip it and log it.

- **RATE LIMIT — mandatory:** CelesTrak updates every 2 hours and enforces one download per update per group; repeated requests return HTTP 403 and repeated errors get the IP firewalled. Therefore:
  - cache each group to `data/raw/celestrak/<group>.csv` with a timestamp;
  - re-download only if the cached file is older than 2 hours;
  - on any non-200 response, **stop**, keep using the cache, and log a warning;
  - commit a snapshot to `artifacts/catalog_snapshot/` so the demo works offline.
- **Optional full catalog:** Space-Track.org (free account) provides the complete GP catalog including all debris. Implement only if time allows, behind a flag.

5.3 Space weather

- Kelvins already includes `F10`, `F3M`, `AP`, `SSN` per CDM — use these for the model.
- For the live view, CelesTrak publishes `https://celestrak.org/SpaceData/SW-All.csv` (daily F10.7, Ap, Kp). Download once, cache, and show the current F10.7 / Ap on the dashboard header ("drag uncertainty: elevated" if Ap ≥ 30 or F10.7 ≥ 200 — label this as a heuristic).

6. Module 1 — Event-level dataset (`features.py`)

Goal: turn the CDM time series into one feature row per event, using only CDMs available at decision time.

Function: `build_event_table(cdm_df, cutoff_days=2.0) -> DataFrame`

For each `event_id`:

1. Sort CDMs by `time_to_tca` descending (oldest first).
2. `target_final_risk` = `risk` of the CDM with the smallest `time_to_tca` (uses the full series — label only).
3. `inputs` = CDMs with `time_to_tca >= cutoff_days`. If none, drop the event for this cutoff.
4. **Training filter (mimic the test set):** keep training events whose last CDM has `time_to_tca < 1`, so the label truly represents near-TCA knowledge.
5. Build features from `inputs`:

Group	Features
Latest CDM	every numeric column of the last input CDM (prefix `last_`)
Trend	`delta_` = last − previous for: `risk`, `miss_distance`, `max_risk_estimate`, `t_sigma_r/t/n`, `c_sigma_r/t/n`
Slope	least-squares slope of `risk` vs `time_to_tca` over inputs (NaN if < 2 CDMs)
Aggregates	`n_cdms`, `risk_max`, `risk_min`, `risk_mean`, `risk_std` over inputs
Timing	`last_time_to_tca`, `span_days` (first − last `time_to_tca`)
Geometry (derived)	`combined_sigma_r = sqrt(t_sigma_r² + c_sigma_r²)` (same for t, n); `mahalanobis_proxy = sqrt((rel_r/σr)² + (rel_t/σt)² + (rel_n/σn)²)`; `log10` of miss distance and sigmas
Categorical	`c_object_type` → LightGBM categorical
Space weather	`last_F10`, `last_F3M`, `last_AP`, `last_SSN`

6. Also store `naive_pred` = `risk` of the last input CDM (the baseline) and `event_id`, `mission_id`.

Splits: `GroupShuffleSplit`-style by `event_id`, 80/20 train/validation, stratified on `is_high_risk` (do it manually: split high-risk events and low-risk events separately, then combine). Fixed seed 42. Held-out test = Kelvins test file (if labels present) else a further 15% of events. Save to `data/processed/events_cutoff{d}.parquet` for cutoffs **2, 3, 4, 5, 7** (lead-time KPI).

7. Module 2 — Models and calibration (`models.py`, `calibrate.py`)

Train on the cutoff-2 table first; repeat for other cutoffs at the end (same code, loop).

7.1 Baseline (must be reported everywhere). `pred = naive_pred` (latest available CDM risk). This baseline was very hard to beat in the original competition — showing we know that earns credibility.

7.2 Regressor — final risk. `LGBMRegressor`, objective `regression` (L2), target `target_final_risk`. Sample weights: 10× for high-risk events (high-risk accuracy matters most). Starting params: `n_estimators=2000, learning_rate=0.03, num_leaves=31, min_child_samples=20, subsample=0.8, subsample_freq=1, colsample_bytree=0.8`, early stopping 100 rounds on validation. Clip outputs to the observed risk range.

7.3 Classifier — is high-risk. `LGBMClassifier`, label `is_high_risk = target_final_risk >= -6`, `scale_pos_weight = n_neg / n_pos`, same params, early stopping on validation AUC-PR.

7.4 Quantile models — confidence band. Two `LGBMRegressor(objective="quantile", alpha=0.1 / 0.9)` for P10 and P90 of final risk; the regressor from 7.2 is the point estimate (P50 in the UI). Enforce `P10 ≤ P50 ≤ P90` by sorting. Report empirical coverage of [P10, P90] on test (target ≈ 80%).

7.5 Calibration. Fit `IsotonicRegression(out_of_bounds="clip")` on the classifier's validation probabilities → `p_high_calibrated`. To avoid calibrating on the same data used for early stopping, split validation 50/50 into `val_es` (early stopping) and `val_cal` (calibration + threshold).

7.6 Threshold for >95% recall. On `val_cal`, choose the largest threshold τ such that recall of high-risk events ≥ 0.95. Save τ in `artifacts/model_config.json`. Report the false-alarm rate at τ (fraction of low-risk events flagged) vs the baseline's false-alarm rate at the same recall (baseline flags events with `naive_pred ≥ some level`; sweep that level to hit 95% recall and compare).

7.7 Optional (only if ahead of schedule): a small GRU/LSTM over the input CDM sequence (pad to the last 8 CDMs, 20 key features, PyTorch, CPU). Stack its output as a feature into LightGBM. Skip RL entirely; mention in the pitch as future work.

8. Module 3 — Evaluation (`evaluate.py`)

Compute on the held-out test set for **baseline vs our model**, write everything to `artifacts/metrics.json` and plots to `artifacts/plots/`:

- **Kelvins score** L = MSE_HR / F2 (lower is better), plus MSE_HR and F2 separately.
- **Recall, precision, false-alarm rate** for the high-risk class at τ.
- **Brier score** of `p_high_calibrated` vs `is_high_risk`; also Brier of the uncalibrated probability and of a baseline probability (logistic fit of `naive_pred`), to show the gain from calibration.
- **Reliability diagram** (10 bins, quantile binning because of class imbalance).
- **Band coverage**: fraction of events with true final risk in [P10, P90].
- **Lead-time curve**: recall at τ and F2 for cutoffs 2, 3, 4, 5, 7 days, model vs baseline. Plot as a line chart.
- **SHAP global importance** (bar chart of mean |SHAP|).

`metrics.json` shape:

```
{
  "cutoff_days": 2,
  "n_test_events": 2167,
  "n_test_high_risk": 150,
  "baseline": {"kelvins_score": 0.0, "mse_hr": 0.0, "f2": 0.0, "recall": 0.0,
               "false_alarm_rate": 0.0, "brier": 0.0},
  "model":    {"kelvins_score": 0.0, "mse_hr": 0.0, "f2": 0.0, "recall": 0.0,
               "false_alarm_rate": 0.0, "brier": 0.0, "brier_uncalibrated": 0.0,
               "band_coverage_p10_p90": 0.0, "threshold": 0.0},
  "lead_time": [{"cutoff_days": 2, "model_recall": 0.0, "baseline_recall": 0.0,
                 "model_f2": 0.0, "baseline_f2": 0.0}],
  "screening": {"objects": 0, "hours": 72, "step_s": 60, "seconds": 0.0}
}
```

(Values above are placeholders; real values come from runs. Never hard-code results.)

Honesty rule: if the model does not beat the baseline on a metric, report it as is. A truthful "we beat it on recall and calibration, tie on MSE" is a stronger pitch than inflated numbers.

9. Module 4 — Explainability and briefing (`explain.py`)

9.1 SHAP. `shap.TreeExplainer` on the classifier (and the regressor). For each event, return the top 5 features by |SHAP| with sign and value. Maintain a `FEATURE_LABELS` dict mapping raw names to plain English, e.g. `last_c_sigma_r` → "Debris radial uncertainty", `delta_miss_distance` → "Change in miss distance since previous CDM", `slope_risk` → "Risk trend over time". Precompute SHAP for all demo events into `artifacts/demo_events.json`.

9.2 Decision logic (`decision.py`).

```
if p_high_calibrated >= τ and p90_risk >= -4:   decision = "MANEUVER"   # Go: plan avoidance
elif p_high_calibrated >= τ:                    decision = "WATCH"      # high-risk class, monitor
else:                                           decision = "HOLD"       # No-Go: no action
```

The −4 level (PoC 10^-4) is the maneuver trigger shown in the brief. Make both thresholds configurable in `config.py`.

9.3 Operator briefing. Input: a JSON with event id, decision, P10/P50/P90 risk, calibrated probability, miss distance, time to TCA, top SHAP drivers with labels, recommended Δv (if computed), and current space weather. Output: exactly three lines.

```
[ALERT] <one sentence: risk level, time to TCA, object type>
[DIAGNOSTIC] <one sentence: the top 1–2 drivers in plain words>
[ACTION] <one sentence: decision and Δv recommendation, or "no action, re-assess at next CDM">
```

- LLM call (if `LLM_PROVIDER=anthropic` and a key is set): system prompt instructs the model to use **only numbers present in the input JSON**, never invent values, ≤ 25 words per line, output exactly the three lines. Timeout 8 s.
- **Template fallback (always available):** fill the same three lines with Python f-strings. Use it when the LLM is disabled, times out or errors. The UI must look identical either way.
- Cache briefings per event id in `artifacts/briefings.json`.

10. Module 5 — Collision probability and maneuver planner (`pc.py`, `cw.py`, `maneuver.py`)

10.1 Collision probability (2D Foster method) — `pc.py`

Inputs (all in the primary's RTN frame): relative position **r** (m, secondary − primary), relative velocity **v** (m/s), position covariances **C_t**, **C_c** (3×3, m²), hard-body radius **R** (m).

1. Build each 3×3 covariance from sigmas and correlations: `C = [[σr², ρtr σr σt, ρnr σr σn], [ρtr σr σt, σt², ρnt σt σn], [ρnr σr σn, ρnt σt σn, σn²]]` using Kelvins columns `x_sigma_r/t/n`, `x_ct_r` (ρtr), `x_cn_r` (ρnr), `x_cn_t` (ρnt).
2. Combined covariance `C = C_t + C_c`.
3. Encounter plane: $\hat{y} = v/|v|$; $\hat{x}$ = unit vector of the component of **r** perpendicular to $\hat{y}$ (if $|r_\perp| \approx 0$, pick any unit vector $\perp \hat{y}$); $\hat{z} = \hat{x} \times \hat{y}$. Projection matrix $P = [\hat{x}; \hat{z}]$ (2×3).
4. 2D mean $\mu = P\,r$ (= $(|r_\perp|, 0)$), 2D covariance $\Sigma = P\,C\,P^T$.
5. $Pc = \iint_{|\xi| \le R} N(\xi; \mu, \Sigma)\, d\xi$. Compute numerically: polar grid over the disk (e.g. 64 radial × 128 angular points) or `scipy.integrate.dblquad`. Vectorize the grid version — it will be called thousands of times by the maneuver search.
6. `R = (t_span + c_span) / 2` when spans are diameters (Kelvins assumes a 2 m minimum for the chaser).
7. Return Pc and log10(Pc) (floor at 1e-30).

Validation: on Kelvins CDMs with complete covariance, compare `log10(Pc_ours)` with the `risk` column. Expect strong rank correlation (Spearman > 0.8) and order-of-magnitude agreement; report the correlation. Differences are expected (ESA uses its own tools and object sizes). This proves the engine is physically sound before we use it for maneuvers.

10.2 Clohessy–Wiltshire relative motion — `cw.py`

Mean motion $n = \sqrt{\mu_E / a^3}$, $\mu_E = 3.986004418 \times 10^{14}\ m^3/s^2$, a = primary semi-major axis (Kelvins `t_j2k_sma` in km → m). Frame: x radial, y along-track, z cross-track. Impulsive Δv at time 0 with zero initial offset; displacement after time t:

- **Along-track burn** (Δv_y): $x(t) = \frac{2}{n}(1 - \cos nt)\,\Delta v_y$, $y(t) = (4\sin(nt)/n - 3t)\,\Delta v_y$.
- **Radial burn** (Δv_x): $x(t) = (\sin(nt)/n)\,\Delta v_x$, $y(t) = \frac{2}{n}(\cos(nt) - 1)\,\Delta v_x$.
- **Cross-track burn** (Δv_z): $z(t) = (\sin(nt)/n)\,\Delta v_z$.

Implement the full CW state-transition matrix (6×6) as a function `cw_stm(n, t)`; the formulas above are special cases and become unit tests. Key intuition for the pitch: for long lead times the along-track term grows as $-3 \cdot \Delta v \cdot t$, so **earlier burns need far less fuel** (0.1 m/s one day early shifts the satellite about 26 km along-track).

10.3 Maneuver search — `maneuver.py`

`search_maneuvers(event, target_pc=1e-6) -> ManeuverResult`

1. Grid: Δv magnitudes `logspace(-3, log10(2), 40)` m/s × directions {+along, −along, +radial, −radial} × burn lead times {6, 12, 24, 36, 48, 72} h before TCA (skip lead times longer than the event's time to TCA when used live).
2. For each option: displacement **δ** of the primary at TCA from `cw.py`; new relative position `r' = r − δ` (primary moved by δ, secondary unchanged); keep **v** and covariances; recompute Pc with `pc.py`.
3. **Pareto front:** for each Δv magnitude, the minimum Pc over directions and lead times. Return all points (for the scatter) and the front.

4. **Recommendation:** the smallest Δv achieving Pc ≤ `target_pc`; tie-break by later burn (keeps options open). If none achieves it, return the option with the lowest Pc and flag `target_met=false`.
5. **Propellant estimate:** `m_prop = m0 (1 − exp(−Δv / (Isp g0)))` with defaults `m0=500 kg`, `Isp=220 s` (hydrazine), `g0=9.80665`. Show grams of fuel in the UI.
6. **Secondary-conjunction check (live mode only):** apply the burn to the primary's state (add Δv in TEME at burn time), propagate with two-body + J2 (`scipy.integrate.solve_ivp`, rtol 1e-9) to TCA + 24 h, and compare against SGP4 positions of the other catalog objects within the altitude band. Report `new_conjunctions` with miss distance < 5 km. In Kelvins (offline) mode, mark this check "not applicable — no catalog context".

Output schema in Section 13 (`/maneuver`). The UI plots residual Pc (log scale) vs Δv with the recommended point highlighted — the same chart as in the brief.

11. Module 6 — Live screener (`screen.py`)

```
screen(primary_norad: int, hours=72, step_s=60, threshold_km=10) -> list[Conjunction]
```

1. Load cached OMM CSV groups (Section 5.2). Build `Satrec` objects; drop objects whose epoch is older than 30 days.
2. **Prefilter by altitude band:** compute perigee/apogee for every object from mean motion and eccentricity; keep objects whose [perigee − 25 km, apogee + 25 km] overlaps the primary's band. This removes most objects.
3. **Vectorized propagation:** `sgp4.api.SatrecArray(sats).sgp4(jd, fr)` over the time grid (TEME, km). Process in chunks to bound memory.
4. **Coarse search:** distances from the primary at every step; keep candidates whose min distance < 50 km.
5. **Refine TCA:** for each candidate, minimize distance with `scipy.optimize.minimize_scalar` (bounded, ±2 steps around the coarse minimum) at 1-second resolution.
6. Report pairs with refined miss distance < `threshold_km`: object ids/names, TCA (UTC ISO), miss distance (km), relative speed (km/s), and relative position in the primary's RTN frame.
7. **Live Pc (clearly labelled "estimated"):** TLE/OMM data has no covariance, so assume position sigmas that grow with data age: `σ_r = 0.1 km + 0.05 km/day`, `σ_t = 0.3 km + 0.3 km/day`, `σ_n = 0.1 km + 0.05 km/day` (age = days since epoch at TCA). State this assumption in the UI. Then reuse `pc.py` and `maneuver.py`.
8. **Throughput benchmark:** log objects screened, time span, step, and wall-clock seconds into `metrics.json["screening"]`. Optional "all-vs-all" mode: per time step, `scipy.spatial.cKDTree(positions).query_pairs(r=10)` on the prefiltered population — report its timing separately.
9. Default primary: ISS (NORAD 25544). Also allow any NORAD id; include a preset list of a few Indian satellites by NORAD id fetched via `CATNR=` (look up ids at build time from the `active` group by name, e.g. names containing "CARTOSAT" or "RISAT"; do not hard-code ids you have not verified).
10. Run the screen once at startup in a background thread and cache results for 2 hours; the API serves the cache.

Frames note: SGP4 outputs TEME. For RTN of the primary: $\hat{R} = r/|r|$, $\hat{N} = (r \times v)/|r \times v|$, $\hat{T} = \hat{N} \times \hat{R}$. For the globe, convert TEME → ECEF using GMST rotation (adequate for display); compute lat/lon/alt with a spherical or WGS-84 approximation.

12. Module 7 — Dashboard (`web/`)

Stack: Vite + React + TypeScript, CesiumJS via `vite-plugin-cesium`, Recharts for charts. **Cesium without a token:** use the bundled Natural Earth II imagery (`TileMapServiceImageryProvider.fromUrl(buildModuleUrl("Assets/Textures/NaturalEarthII"))`) and no terrain, so the app works offline and without an Ion account. If Cesium setup takes over an hour, fall back to `globe.gl` with the same data.

Theme: dark "mission control" — near-black background, monospace numbers, amber for warnings, red for MANEUVER, teal for HOLD. Large readable type for projector demos.

Pages / panels:

1. **Mission Control (home)**
   - Header: OrbitGuard logo, UTC clock, space-weather chip (F10.7, Ap), data source toggle **[Historical (ESA CDMs) | Live (CelesTrak)]**.
   - Left: 3D globe. Live mode: catalog objects as small points (grey = debris, white = payloads, primary in cyan); conjunction partners in red with a line to the primary at TCA; primary's orbit path for the next 90 min.
   - Right: **Ranked alert list** — sorted by calibrated probability then P50 risk. Each row: decision badge (MANEUVER / WATCH / HOLD), event id or object pair, time to TCA, P50 risk as "1 in N", P10–P90 band as a small bar.
2. **Alert detail** (click a row)
   - **CDM history chart:** risk (log10) vs time to TCA, with the 10^-4 trigger line and the 2-day cutoff marked; the model's P10–P90 band shown at TCA as a shaded box.
   - **Replay slider:** drag "now" from 7 days to 2 days before TCA; the prediction, band, decision and SHAP update using the matching cutoff model (precomputed for cutoffs 7, 5, 4, 3, 2). This visualizes the lead-time KPI.

- **Why flagged:** horizontal SHAP bar chart (top 5, plain-English labels).
- **Operator briefing:** the three ALERT / DIAGNOSTIC / ACTION lines in a terminal-style box.
- **Maneuver panel:** Pc vs Δv scatter (log y), Pareto front line, recommended burn highlighted, text: “Recommended: +0.12 m/s along-track, 24 h before TCA → Pc 3e-7 (≈ 27 g propellant)”.
- **Export report** button: downloads a JSON/printable HTML with all of the above (regulator persona).

3. **Metrics** — tiles for Kelvins score, recall, false-alarm rate, Brier (model vs baseline); reliability diagram; lead-time curve; screening throughput. Data from `/api/metrics`.

Rules: all data through `web/src/api.ts`; one TypeScript type per Section 13 schema; loading and error states for every request; the UI must render from mock JSON alone.

13. API contract (FastAPI, `api/main.py`)

Base URL `http://localhost:8000/api`. CORS open for `http://localhost:5173`. All times UTC ISO-8601. Risk values are log10 PoC.

Method & path	Returns
`GET /health`	`{"status":"ok","model_loaded":true,"catalog_age_hours":1.2}`
`GET /alerts?source=historical&limit=50`	ranked list (below)
`GET /alerts/{event_id}?cutoff=2`	alert detail (below)
`GET /alerts/{event_id}/maneuver`	maneuver result (below)
`GET /live/catalog?limit=3000`	`{"epoch":"...","objects":[{"norad":25544,"name":"ISS (ZARYA)","type":"PAYLOAD","lat":0,"lon":0,"alt_km":420}]}`
`GET /live/conjunctions?norad=25544&hours=72`	`{"primary":{...},"conjunctions":[{"secondary":{...},"tca":"...","miss_km":3.2,"rel_speed_kms":11.8,"pc_est":1e-6,"rtn_km":[0.1,3.1,0.4]}],"compute_seconds":42.0}`
`GET /live/orbit/{norad}?minutes=90&step_s=30`	`{"points":[{"t":"...","lat":0,"lon":0,"alt_km":0}]}`
`GET /metrics`	contents of `artifacts/metrics.json`
`GET /space-weather`	`{"date":"...","f107":150.2,"ap":12,"heuristic_drag_flag":"normal"}`

Alert list item:

```
{
  "event_id": "1234",
  "source": "historical",
  "decision": "MANEUVER",
  "time_to_tca_days": 2.1,
  "p_high": 0.91,
  "risk_p10": -5.2, "risk_p50": -4.1, "risk_p90": -3.3,
  "baseline_risk": -4.6,
  "miss_distance_m": 180.0,
  "chaser_type": "DEBRIS"
}
```

Alert detail = list item plus:

```
{
  "cdm_history": [{"time_to_tca": 6.8, "risk": -7.1, "miss_distance_m": 900.0}],
  "available_cutoffs": [7, 5, 4, 3, 2],
  "shap_top": [{"feature": "last_c_sigma_r", "label": "Debris radial uncertainty",
                "value": 120.5, "shap": 0.84}],
  "briefing": ["[ALERT] ...", "[DIAGNOSTIC] ...", "[ACTION] ..."],
  "briefing_source": "llm",
  "true_final_risk": -3.9
}
```

(`true_final_risk` only for historical events, shown in the UI as “actual outcome” for demo credibility.)

Maneuver result:

```
{
  "event_id": "1234",
  "target_pc": 1e-6,
  "target_met": true,
  "pre_burn_pc": 7.9e-5,
  "recommended": {"dv_ms": 0.12, "direction": "+along", "burn_lead_h": 24,
                  "post_pc": 3.1e-7, "propellant_g": 27.8},
  "points": [{"dv_ms": 0.01, "direction": "+along", "burn_lead_h": 6, "post_pc": 6e-5}],
  "pareto": [{"dv_ms": 0.01, "post_pc": 4e-5}],
  "secondary_check": {"applicable": false, "new_conjunctions": []}
}
```


Demo data: at startup, load `artifacts/demo_events.json` — about 30 held-out test events (all high-risk ones the model catches, a few it misses, and ~15 low-risk) with predictions for every cutoff, SHAP, briefings and maneuvers precomputed. The API serves these without recomputation, so the demo is instant and offline-safe. A script `python -m orbitguard.build_demo` creates the file.

14. Tests (`tests/`, run with `pytest`)

1. **Pc, centred isotropic case:** μ = 0, Σ = σ²I → `Pc = 1 − exp(−R² / (2σ²))`. Check within 1e-3 relative for σ = 100 m, R = 5 m and σ = 10 m, R = 10 m.
2. **Pc, far miss:** μ = (10 km, 0), σ = 50 m → Pc < 1e-20.
3. **CW:** for along-track Δv = 0.1 m/s and t = 86400 s, `y(t)` ≈ −3·Δv·t within 2% (LEO, a = 6,778 km); `cw_stm(n, 0)` = identity.
4. **Maneuver monotonicity:** for a synthetic head-on conjunction, larger Δv never increases the best achievable Pc on the Pareto front.
5. **No leakage:** for every event row at cutoff d, all CDMs used have `time_to_tca ≥ d`; no `event_id` appears in more than one split.
6. **Baseline sanity:** the naive baseline's Kelvins score on validation is finite and recomputable.
7. **API contract:** every endpoint returns JSON that validates against `api/schemas.py`, in both mock mode and real mode.
8. **CelesTrak cache:** a cached file younger than 2 hours is never re-downloaded (mock `requests.get` and assert no call).

15. Build order (solo developer + Claude Code)

Hackathon: **9–11 October 2026.** One developer working with Claude Code. Build strictly in this order. Each milestone is small, ends in a runnable state, and must pass its checks before the next one starts. Report results to the developer at the end of every milestone and wait for a go-ahead.

Milestone 0 — Skeleton + data (Day 1, ~2 h)

- Repo layout (Section 4), `requirements.txt`, `Makefile`, `.env.example`, `CLAUDE.md`, `.gitignore`, `git init`.
- `download.py`: Kelvins data, CelesTrak cached snapshot, SW-All.csv (all cached, Section 5).
- `api/schemas.py` + mock JSON for every endpoint (Section 13); FastAPI serving mocks.
- Vite + React shell that renders the mock alert list and a Cesium globe (no token).
- **Done when:** `make api` and `make web` show a globe and a mock alert list; data files exist in `data/raw/`.

Milestone 1 — ML core (Day 1, ~3 h)

- `features.py` for cutoff 2 days; splits; test 5 (leakage).
- Naive baseline; LightGBM regressor + classifier; Kelvins score, F2, recall, MSE_HR printed for baseline vs model.
- **Done when:** a table of baseline vs model metrics prints; tests pass.

Milestone 2 — Trust layer (Day 1 night, ~3 h)

- Quantile models (P10/P90), isotonic calibration, threshold τ for ≥95% recall, Brier, reliability diagram.
- Cutoffs 3, 4, 5, 7 → lead-time curve. SHAP global + per event. `evaluate.py` writes `metrics.json` and plots.
- `decision.py`, template briefings, `build_demo.py` → `demo_events.json`. Real `/alerts`, `/alerts/{id}`, `/metrics` endpoints.
- **Done when:** the UI's alert list shows real ranked Kelvins events from `demo_events.json`.

Milestone 3 — Physics (Day 2 morning, ~3 h)

- `pc.py` + tests 1–2; Spearman validation vs Kelvins `risk`, written to `metrics.json`.
- `cw.py`, `maneuver.py` + tests 3–4; `/alerts/{id}/maneuver`; maneuvers precomputed into `demo_events.json`.
- **Done when:** maneuver JSON is served for every MANEUVER/WATCH demo event; all tests pass.

Milestone 4 — Dashboard, historical mode = MVP (Day 2 afternoon, ~4 h)

- Alert detail page: CDM history chart with band, SHAP chart, briefing box, maneuver chart, replay slider.
- Metrics page. Mission-control styling.
- **Done when:** the full historical demo (Section 16, steps 3–6) runs offline. **This is the minimum submittable product — commit and tag it `mvp`.**

Milestone 5 — Live mode (Day 2 evening, ~3 h)

- `screen.py` for the ISS vs cached catalog, estimated Pc, throughput benchmark into `metrics.json`.
- `/live/*` endpoints; globe live mode (catalog points, primary orbit path, conjunction lines); space-weather chip.
- **Done when:** live mode works from the cached catalog with the internet off.

Milestone 6 — Polish (Day 2 night, ~2 h)

- LLM briefing with template fallback; export report; secondary-conjunction check (if time).
- README with run instructions and demo script.

Milestone 7 — Freeze (Day 3 morning)

- Feature freeze. Bug fixes only. Run all tests; regenerate `metrics.json` and plots; copy numbers into the deck from `metrics.json` only.
- Record a 2-minute backup video of the demo. Rehearse the pitch 2–3 times.

Time-saving rules for a solo build:

- Run long jobs (training all cutoffs, SHAP, catalog screening) as background scripts that save to `artifacts/`; the API only reads artifacts.
- Reuse precomputed results; never compute in a request handler what can be computed once.
- If a milestone runs more than 50% over its time, cut from the list below and move on.

Cut list (drop in this order if behind): LSTM → all-vs-all screening → secondary-conjunction check → Space-Track integration → export report → LLM briefing (keep template) → replay slider → live mode (show screening as a recorded result instead).
Never cut: baseline comparison, calibration + recall metrics, SHAP, maneuver chart, template briefing, historical dashboard.

16. Demo script (≈ 5 minutes)

1. **Hook (30 s):** “It’s 2 a.m. An operator has 300 alerts this week; 99% are noise. One is real. Which one?”
2. **Live sky (45 s):** Mission Control in live mode — the real catalog around the ISS; “we screened N objects over 72 hours in X seconds” (from `metrics.json`).
3. **Triage (60 s):** switch to historical ESA events. The ranked list puts real threats on top. Open a MANEUVER alert.
4. **Trust (60 s):** CDM history with the confidence band, SHAP drivers, the three-line briefing. Drag the replay slider: the model flagged it N days before TCA.
5. **Action (45 s):** maneuver chart — “+0.12 m/s along-track 24 h out brings risk below 1 in a million, using about 28 g of fuel. The same burn 6 h out would need several times more.”
6. **Proof (45 s):** metrics page — recall, false-alarm rate, Brier and Kelvins score vs the ESA naive baseline, lead-time curve. Be honest about where we tie.
7. **Close (15 s):** “OrbitGuard turns noisy alerts into calibrated, explained, fuel-aware decisions — built on real ESA data and the live US catalog. Next: covariance-aware live data, RL maneuver planning, ISRO operator workflows.”

17. Known limitations (say them before the judges do)

- Live catalog data has no covariance, so live Pc uses assumed uncertainties; the trained risk model runs on ESA CDMs, which do carry covariance. The two layers are screening (live) and risk assessment (CDM-based).
- The Kelvins data covers a handful of ESA missions from 2015–2019; generalization to other operators is untested.
- Clohessy–Wiltshire is linear and assumes near-circular orbits and short-to-medium lead times; it is suitable for ranking burn options, not for flight-ready planning.
- High-risk events are rare, so metrics on them have wide uncertainty; report the number of high-risk test events next to every metric.

18. References

- ESA Kelvins Collision Avoidance Challenge — challenge, data and scoring pages: `kelvins.esa.int/collision-avoidance-challenge/`
- Dataset: Zenodo record 4463683.

- Uriot et al., “Spacecraft Collision Avoidance Challenge: design and results of a machine learning competition”, arXiv:2008.03069.
- CelesTrak GP data formats and usage policy: `celestrak.org/NORAD/documentation/gp-data-formats.php`
- Vallado et al., “Revisiting Spacetrack Report #3” (SGP4); Python `sgp4` package documentation.
- Foster & Estes (1992) / Alfano: 2D collision probability in the encounter plane.
- Clohessy & Wiltshire (1960): terminal guidance / relative motion equations.